Лабораторная работа № 4

Вычисление наибольшего общего делителя

Шуваев С. А.

Российский университет дружбы народов, Москва, Россия

Информация

Докладчик

- Шуваев Сергей Александрович
- студент
- Российский университет дружбы народов
- 1032262691@pfur.ru
- https://Grinders060050.github.io/ru/



Цель работы

Реализовать все рассмотренные алгоритмы программно.

Задание

1. Алгоритм Евклида.
2. Бинарный алгоритм Евклида.
3. Расширенный алгоритм Евклида.
4. Расширенный бинарный алгоритм Евклида.

Выполнение лабораторной работы

```julia
function euclid_gcd(a::Integer, b::Integer)
    @assert 0 < b < a "Требуется 0 < b < a"
    r_prev, r = a, b          # r_{-1} = a, r_0 = b
    while r != 0              # шаг 2–3: пока остаток не станет 0
        r_prev, r = r, r_prev % r
    end
    return r_prev             # это и есть НОД
end
```

Figure 1: Алгоритм Евклида

Выполнение лабораторной работы

```julia
function binary_gcd(a::Integer, b::Integer)
    @assert 0 < b < a "Требуется 0 < b < a"

    # Свойство 1: выносим общую степень двойки
    k = 0
    while iseven(a) && iseven(b)
        a >>= 1
        b >>= 1
        k += 1
    end

    while b != 0
        # Свойство 2: убираем двойки из каждого числа отдельно
        while iseven(a); a >>= 1; end
        while iseven(b); b >>= 1; end

        # Свойства 3 и 4: оба нечётны — сводим к разности,
        # сохраняя a ≥ b, чтобы вычитание шло в одну сторону
        if a > b
            a, b = b, a
        end
        b -= a             # b становится чётным (пока не станет 0)
    end

    return a << k          # возвращаем вынесенную степень двойки
end
```

```
binary_gcd (generic function with 1 method)
```


Figure 2. Бинарный алгоритм Евклида

```julia
function extended_gcd_std(a::Integer, b::Integer)
    @assert 0 < b <= a
    old_r, r = a, b
    old_s, s = 1, 0
    old_t, t = 0, 1
    while r != 0
        q = old_r ÷ r
        old_r, r = r, old_r - q * r
        old_s, s = s, old_s - q * s
        old_t, t = t, old_t - q * t
    end
    return old_r, old_s, old_t   # d, x, y
end
```

```
extended_gcd_std (generic function with 1 method)
```

Figure 3: Расширенный алгоритм Евклида.

Выполнение лабораторной работы

```
function extended_binary_gcd(x::Integer, y::Integer)
    @assert x > 0 && y > 0 "Требуется x > 0 и y > 0"

    # шаги 1–2: выносим общую степень двойки
    g = one(x)
    while iseven(x) && iseven(y)
        x >>= 1;  y >>= 1;  g <<= 1
    end

    u, v = x, y
    A, B = one(x), zero(x)   # u = A*x + B*y
    C, D = zero(x), one(x)   # v = C*x + D*y

    while u != 0
        # пока u чётно — делим u на 2, корректируя (A, B)
        while iseven(u)
            u >>= 1
            if iseven(A) && iseven(B)
                A >>= 1;  B >>= 1
            else
                A = (A + y) >> 1
                B = (B - x) >> 1
            end
        end

        # пока v чётно — делим v на 2, корректируя (C, D)
        while iseven(v)
            v >>= 1
            if iseven(C) && iseven(D)
                C >>= 1;  D >>= 1
            else
```

Figure 4: Расширенный бинарный алгоритм Евклида.

Выводы

В результате выполнения данной лабораторной работы я программно на практике реализовал 4 Евклидова Алгоритма в среде Julia.

Список литературы